

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:50

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANALYTICAL PROBLEM SOLVING

- 1** Trace the input expression $x = a + b * (c - d) / e$ through all phases of a compiler by identifying the tokens generated during lexical analysis, constructing the syntax tree during parsing, generating intermediate code, applying possible optimizations, and interpreting the relevance of the final target code produced.
- 2** Analyze the tokenization process of the expression `int value = count++--num;` by identifying ambiguities in operator recognition, explaining how a LEX-based lexical analyzer processes such patterns, modifying the rules to resolve conflicts, and justifying the correctness of the updated lexical specification.
- 3** Determine the total number of tokens present in the given C program that includes the declaration of the `main()` function, variable initialization, conditional statement, and return statements, while carefully considering how the lexical analyzer ignores comments (`/* Find max of a and b */`) and whitespace. Analyze each line of the program to identify keywords, identifiers, operators, constants, and punctuation symbols as individual tokens, and compute the final token count based on proper lexical classification.
- 4** Analyze the performance of a lexical analyzer in a hypothetical programming language where the lexer processes up to 1,000,000 characters per second, with each token having an average length of 8 characters. Considering that the deterministic finite automaton (DFA) recognizes 20 different token types and performs an average of 1.5 state transitions per character, determine the total number of tokens generated, estimate the total number of DFA state transitions required, and interpret how efficiently the lexical analysis phase operates for the given source code size.
- 5** Analyze the working of a lexical analyzer for a simplified programming language that includes keywords (`if`, `else`, `while`, `return`, `int`, `float`), operators (`+`, `-`, `=`, `==`, `<`, `>`), separators (`(`, `)`, `{`, `}`, `;`), identifiers (a sequence of letters followed by digits), integer literals, and floating-point literals. For the input string `int var123 12345; float var456 = 123.45; if (var123 == var456) { return; }`, determine the total number of tokens generated by the lexical analyzer and provide a categorized list of tokens based on their types such as keywords, identifiers, operators, separators, and literals, ensuring proper classification and interpretation of each token..

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

